TTT4295 - Acoustic Signal Processing
Assignment 3 - Physical modeling of musical instruments

Peter Svensson
Sara Martin

October 13, 2026

Your deliverable should include the following:

- One plot of the output "pulse train" signal for one note (e.g., A = 440 Hz) from Task 1, indicating the delay N in samples.
- One audio file of a plucked note using your best version of the synthesizer, combining the fractional-delay filter, the frequency-dependent loop filter, and a noise-burst source signal (Tasks 2, 3 and 5).
- One plot of the FFT spectrum showing suppressed harmonics when the string is excited at the center (Task 4).
- One plot of the extracted source signal from the inverse filtering of *TTT4295 Assignment3 soundfile1.mat* (Task 8).

If you get this assignment for your report, then all tasks (except those marked as optional) must be included in the report.

Introduction

In this assignment, you will synthesize the sound from a one-dimensional music instrument, particularly those produced by vibrating structures, such as strings, tubes, and bars, using a very efficient and popular approach called *1D Digital Waveguide synthesis*. Here is a concise summary of that method.

The 1D Digital Waveguide model (DW) is based on the following main concepts:

- The method simulates how waves propagate and reflect within a one-dimensional medium (e.g., a vibrating string or air column).
- A 1D waveguide models how sound waves travel back and forth along a one-dimensional path, representing the physical structure of an instrument.
- The model describes two traveling waves: leftward and rightward propagating waves. These waves represent pressure variations or displacement traveling along the length of the instrument.

This technique uses different digital components:

- **Delay Lines:**
  These simulate the time it takes for waves to travel across the instrument's length. The delay is directly proportional to the instrument's length and inversely proportional to the wave propagation speed.
- **Reflection and Transmission Filters:**
  These mimic how waves reflect or transmit at boundaries or junctions. For example, a fixed string end reflects waves with a phase change, while a tube's open end might reflect less energy back to the instrument.
- **Loss and Dispersion Filters:**
  Real instruments lose energy due to friction, air resistance, and other factors. Filters simulate energy loss and frequency-dependent effects (dispersion) to better match the timbre of real instruments.

In this assignment, you will implement these different components with different approaches, more specifically:

- A fractional-delay filter
- A more realistic (frequency-dependent) loop filter
- Source position filter
- Source signal
- Radiation filters

A fractional-delay filter:

The most basic implementation of the 1D DW limits the delay lines to an integer number of samples. As a consequence of that limitation, not all desired pitches (or fundamental frequencies) can be generated. Therefore, fractional delay lines are commonly used to overcome this fact.

In Ref. [1], it is shown that a fractional delay filter can be implemented as a single-parameter filter, with

$$H_{delay}(z) = \frac{C + z^{-1}}{1 + Cz^{-1}} \tag{1}$$

This is an all-pass filter, meaning that the corresponding frequency response is flat. We can confirm that by replacing z^{-1} by $e^{-j\omega T}$, where $T = 1/f_S$:

$$H_{delay}(\omega) = \frac{C + e^{-j\omega T}}{1 + Ce^{-j\omega T}}$$

$$\Rightarrow |H_{delay}(\omega)|^2 = \frac{|C + \cos\omega T + j\sin\omega T|^2}{|1 + C\cos\omega T + jC\sin\omega T|^2} = \frac{C^2 + 2C\cos\omega T + 1}{1 + 2C\cos\omega T + C^2} = 1$$

But, what is the effective delay? We find that from the slope of the phase function if that slope is linear for low frequencies (since a linear phase, as a function of frequency, corresponds to a delay). So, the phase function is, with Num.$[H_{delay}]$ referring to the *Numerator* of H_{delay} and Denom.$[H_{delay}]$ referring to the *Denominator* of H_{delay},

$$\begin{aligned}
\arg(H_{delay}(\omega)) &= \arg(\text{Num.}[H_{delay}(\omega)]) - \arg(\text{Denom.}[H_{delay}(\omega)]) \\
&= \arctan\frac{\Im[\text{Num.}[H_{delay}(\omega)]]}{\Re[\text{Num.}[H_{delay}(\omega)]]} - \arctan\frac{\Im[\text{Denom.}[H_{delay}(\omega)]]}{\Re[\text{Denom.}[H_{delay}(\omega)]]} \\
&= \arctan\frac{\sin\omega T}{C + \cos\omega T} - \arctan\frac{C\sin\omega T}{1 + C\cos\omega T}
\end{aligned}$$

In the low-frequency limit, we can use series-expansion expressions:

$$\arctan x \approx x, \sin x \approx x, \cos x \approx 1,$$

and then get:

$$\arg(H_{delay}(\omega)) \approx \frac{\omega T}{C+1} - \frac{C\omega T}{1+C} = \omega T\frac{1-C}{1+C} = \omega T_{effective}$$

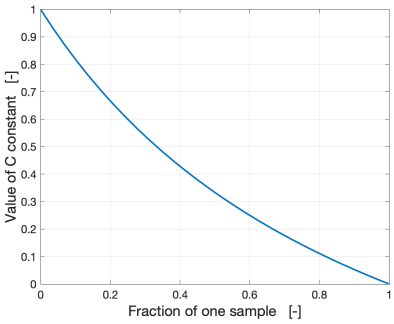


Figure 1: The C coefficient in the filter in Eq.1 needed to get a certain fraction-of-a-sample delay.

T is one sample's delay, so if we manage to achieve $T_{effective} < T$, then we have got an effective delay less than one sample. So, if we want $N_{delay} \in [0, 1]$, we should choose C to fulfill:

$$N_{delay} = \frac{1 - C}{1 + C} \Rightarrow C = \frac{1 - N_{delay}}{1 + N_{delay}}$$

Figure 1 shows the C-values needed to get a certain fraction-of-a-sample delay. The filter in Eq.1 is stable for values of $|C| \leq 1$, [1], so it seems to work with the range of values plotted in Figure 1.

We can check the two extreme values $C = 0$ and $C = 1$, respectively in Eq. 1.

$$C = 0 \Rightarrow H_{delay}(z) = \frac{0 + z^{-1}}{1 + 0 \cdot z^{-1}} = z^{-1}, \text{i.e., indeed one sample}'\text{s delay}$$

$$C = 1 \Rightarrow H_{delay}(z) = \frac{1 + z^{-1}}{1 + z^{-1}} = 1, \text{i.e., indeed zero delay}$$

Tasks

1. **The simplest 1D Digital Waveguide**

   The pulse train generation

   The simplest possible 1D Digital Waveguide simulates two waves, one propagating in the positive x-direction, and the other one in the negative x-direction. If you input a pulse, a "pulse train" with slowly reducing amplitude should be generated as the output signal.

   In this exercise, you should implement this simple version of the 1D-waveguide in Matlab (or in Python or...). Choose the delay, n, as an integer number of sample

steps, for a sampling frequency of $f_S = 44.1\,\text{kHz}$. Set the reflection coefficient at one end to 0.98 (and 1.0 at the other end).

You should then plot the output signal obtained, indicating the delay chosen in number of samples.
Note: Make sure that your output signal has died out to an amplitude that is at least 40 dB down from the initial amplitude.

Hint: One straightforward way to implement this is as an "all-pole filter" in the form of an IIR filter. There are more ways to do this, though.

Synthesis of the musical notes in one octave

Now, generate a set of such "pulse trains" with fundamentals that represent one octave of semitones (by changing the delay of the waveguide, in samples n), from C to H (or B, if you are used to that notation). Include the A with a fundamental frequency of 440 Hz. How large will the errors be, in percent of a semi-tone, for the pitch compared to the theoretical pitches?
Plot the twelve output signals, "pulse trains", and present the fundamental frequencies of the twelve pulse trains as well as the errors, in percent of a semi-tone, for the pitches obtained compared to the theoretical ones.

Submit the generated output signals in one audio file adding half a second of silence in between signals.

Hint: If you derive an expression for the relationship between fundamental frequency and the delay in whole number of samples, then you could easily generate a set of filters for the desired frequencies.

Hint: A semitone is a ratio of the 12th root of 2 to 1.

Generating a melody

As an optional task, choose some simple melody and generate a script that plays it, at a tempo of your choice. NB! Make sure that the decay of each tone is included, even if the next tone is started before the end of a decay! That is, you might need to add the start of the next tone to the decay of the previous tone.
Submit the audio file with the composed melody composed.

2. A more accurate delay filter: the fractional-delay filter

You learned in the previous task that the integer-delay loop filter gives a small but non-negligible error for the fundamental frequency. In the Introduction, it is

described how this can be implemented as a single-parameter, one-pole, one-zero filter, which gives an effective delay between N and $N+1$ by varying C between 0 and 1.

$$H_{delay}(z) = z^{-N}\frac{C+z^{-1}}{1+Cz^{-1}}$$

If your improved delay filter is implemented as the filter above, derive what your entire closed-loop filter will be, on the form

$$H_{closed\ loop} = \frac{B(z)}{A(z)}$$

Hint: For the simple integer-delay loop in task 1, the closed loop transfer function (for a delay by N samples) was $H_{\text{closed loop}} = \frac{B(z)}{A(z)} = \frac{1}{1-0.98z^{-N}}$

Implement this improved delay-filter and present the new pitch errors. Make a comparison with the errors obtained with this implementation and the ones obtained in Task 1.

3. A frequency-dependent loop filter

As described in the slides from the lectures, a first-order low-pass filter with a cut-off frequency of f_0 can be implemented as a single-parameter one-pole, one-zero filter,

$$H_{\text{lowpass}}(z) = \frac{1+z^{-1}}{(1+a)+(1-a)z^{-1}} = \frac{1}{1+a}\frac{1+z^{-1}}{1+\frac{1-a}{1+a}z^{-1}}$$

where $a = \frac{2}{T\omega_0} = \frac{f_S}{\pi f_0}$.

Implement this low-pass filter and play around with values of a that you think give a nicer sound.

Submit an audio file with one of your preferred sounds, indicating to which cut-off frequency it corresponds to.

4. A source position filter

As described in the slides from the lectures, when a string is struck/hit at one particular position, then certain vibration modes will be suppressed. For instance, when the center point of the string is struck, all even harmonics are suppressed. When the string is excited at a position 0.1 of the length, every tenth harmonic will be suppressed etc. Such a filter can be implemented with one zero, the position of which is determined by the excitation position.

$$H_{sourceposition}(z) = 1 - z^{-N_{position}}$$

where $N_{position} = 2 * N_{\text{excitationpoint}} - 1$ and $N_{\text{excitationpoint}}$ is the position for the excitation point, expressed in number of time samples travel time from the end of the string.

Implement such a source-position filter, and demonstrate, by taking an FFT of the output signal, that some harmonics are suppressed. Present the spectrum of the output filter for two different excitation positions.

5. **A source signal**

The original Karplus and Strong paper, which introduced this delay loop to efficiently simulate a 1D-resonator, used a short noise burst as a source signal rather than a single pulse.

Make some examples of noise bursts rather than single pulses as source signals. Try, e.g., with a 100-sample sequence of Gaussian noise. You can also try with a low-pass filtered version of a noise signal, and a simple way of doing this is with Matlab's `cumsum` function:

```
filterednoisesig = cumsum(noisesig);
```

Submit an audio file with the output signal after using a different source signal than a single pulse. Specify which input signal was chosen.

6. **Identify the delay of the loop filter**

In this task, you should explore possibilities to use *inverse filtering* to extract the unknown input/source signal if you know only the output signal. We will make it easy for ourselves since we will know which structure the system has, and therefore it is easier to develop a model of the system - followed by the construction of an inverse filter.

Download the file `TTT4295_Assignment3_soundfile1.mat`, which contains the variables `sig` and `fs`. You happen to know that this signal was generated by a simple delay loop with a frequency-independent attenuation.

Determine how many samples delay the loop filter has.

Hint: The easiest way to identify if a signal contains delayed copies of an input signal is to use the *autocorrelation* function, $r_{xx}(n)$. This function tells how similar a function is to itself, when it is delayed by an amount of n samples: a high value of $|r_{xx}|$ for sample n indicates a high degree of similarity when the signal is shifted by n samples. The Matlab function `xcorr(x,y)` gives the *cross-correlation* between two signals, `x` and `y`, but if `y` = `x`, then we get the autocorrelation!

Plot the correlation obtained proving the number of sample delays of the filter.

7. **Identify the decay of the loop filter**

   Determine the loop scale factor, that is, which factor is the signal attenuated by, for one round?

   Hint: Since the excitation signal was not a pulse in this case, you can not simply check how much each pulse is attenuated from one round to the next. But, if you plot the signal on a logarithmic scale, you should see that the logarithm of the signal amplitude is following a linear decay, which means that the signal amplitude follows an exponential decay, which can be converted to a scale for each round in the loop!

8. **Extract the source signal**

   Once you have identified the delay and loop scale factor in the previous task, construct an inverse filter, filter the signal `sig` with this filter and plot the resulting signal.

   Hint 1: Your loop filter has the form $H_{closed\ loop}(z) = \dfrac{B(z)}{A(z)}$ so an inverse filter should simply be $H_{inverse_filter}(z) = \dfrac{A(z)}{B(z)}$!

   Hint 2: You know that you have found a good model of your system when the signal which comes out of the inverse filter is very short in length!

References

[1] D. A. Jaffe and J. O. Smith, "Extensions of the Karplus-Strong Plucked-String Algorithm," *Computer Music J.* 7, pp. 56-69 (1983).